

Port Scanner: Complete Step-by-Step Guide

Set up, run and understand the project, from opening a terminal to reading the results

Part A Terminal basics | Part B Set up | Part C Run the scanner | Part D Demos | Part E How it works on a network | Part F Troubleshooting

Only scan hosts you own or have explicit permission to scan. Scanning others without permission can be illegal, even for "just testing."

What this project does

Port Scanner is a small Python program that checks which network ports are open on a computer or device. For every open port it also reads a short greeting (a "banner") from the service, which shows what software is running. It uses only Python's standard library, so nothing needs installing except Python itself.

What you need

- A computer running Windows, macOS or Linux.
- Python 3 (Step 1 shows how to check for it and install it).
- The project folder containing `port_scanner.py` (Step 2).
- A terminal, which is a window where you type commands (Part A).
- Only devices you own or have permission to scan.

Part A: Terminal basics

A terminal is a window where you type a command and press **Enter**. The computer runs the command and prints the result. Windows calls its terminals PowerShell and Command Prompt (`cmd`). Both work for everything in this guide.

A1. Open a terminal

System	How to open it
Windows	Open the Start menu, type PowerShell (or cmd) and press Enter.
Windows, straight into a folder	Open the folder in File Explorer, click the address bar, type powershell and press Enter. Or, on Windows 11, right-click an empty area and choose Open in Terminal . The terminal starts inside that folder.
macOS	Press Cmd+Space, type Terminal and press Enter.
Linux	Press Ctrl+Alt+T, or search your apps for Terminal.

A2. Read the prompt

The line where you type is the **prompt**. On Windows it looks like this:

```
PS C:\Users\you\Documents\Network port scanner>
```

The text before the `>` is the folder the terminal is currently in. Commands run in that folder, so the terminal must be inside the project folder to find `port_scanner.py`.

A3. The commands you need

Command	What it does
<code>cd "C:\path\to\Network port scanner"</code>	Go into the project folder. Keep the quotes, because the name has spaces. In Command Prompt, use <code>cd /d</code> if the folder is on another drive.
<code>dir</code> (Windows) or <code>ls</code> (Mac, Linux)	List the files in the current folder.
<code>python --version</code>	Show which Python version is installed.
<code>python port_scanner.py</code>	Run the scanner.
<code>python -m unittest -v</code>	Run the project's tests.
<code>cls</code> (Windows) or <code>clear</code> (Mac, Linux)	Clear the screen.
Ctrl+C	Stop a program that is running.

Typing tips. Press the Up arrow to bring back your previous command. Paste with Ctrl+V or a right-click. If Ctrl+C does nothing, click inside the terminal first (with text selected, Ctrl+C copies instead of stopping). Closing the terminal tab always stops whatever is running.

Part B: Set up

Step 1: Check that Python 3 is installed

```
python --version
```

It should print **Python 3** followed by a version number. On Mac and Linux use `python3 --version`, and type `python3` instead of `python` in every command in this guide.

If Windows says "python is not recognized": go to python.org/downloads, download Python 3 and run the installer. On its first screen tick **Add python.exe to PATH**, then click Install Now. Close your terminal, open a new one and run `python --version` again.

Step 2: Get the project folder

If you received the project as a ZIP file, right-click it and choose **Extract All**. If you use Git, clone it (this needs Git installed and access to the repository):

```
git clone https://github.com/BhonePyaeMin/network-port-scanner.git
```

Either way you end up with a folder that contains `port_scanner.py`, `test_port_scanner.py` and `README.md`.

Step 3: Open a terminal inside the project folder

Use the "straight into a folder" method from A1, or type `cd` and the folder path. To copy the path, open the folder in File Explorer, click the address bar and press Ctrl+C. In the terminal type `cd "`, paste with Ctrl+V, type `"` and press Enter. Then check you are in the right place:

```
dir
```

The list must include `port_scanner.py`.

Step 4: Check that everything works (run the tests)

```
python -m unittest -v
```

After about 2 seconds you should see **Ran 10 tests** and **OK**. The tests start tiny servers on your own computer and check that the scanner finds them, reads their banners and ignores closed ports. If they pass, Python and the project are set up correctly.

Part C: Run the scanner

Step 5: Start the scanner

```
python port_scanner.py
```

Step 6: Answer the three prompts

Type a value and press Enter after each prompt. The value in [brackets] is the default: press Enter without typing to use it.

Prompt	Meaning	Default
Target host/IP	The computer to scan, as an IP address or a name such as 127.0.0.1 or scanme.nmap.org.	127.0.0.1
Start port	First port to check (1 to 65535).	1
End port	Last port to check (1 to 65535).	1024

If you type something that is not a valid port, the scanner asks again.

Step 7: Wait, then read the results

Open ports print as they are found. The time depends on the target and the port range: each closed port can take up to half a second (the connection timeout), so the full default range may take several minutes. A pause between results is normal. Example output (yours will differ):

```
Scanning 127.0.0.1 (127.0.0.1), ports 7995-8005...

8000/tcp open HTTP/1.0 200 OK

Done: 1 open port(s) in 6.1s.
```

Each line shows the **port number**, the protocol (tcp), the state and the **banner**. A blank banner means the service sent nothing. Ports that are not listed are closed or blocked by a firewall. The Done : line gives the number of open ports and the elapsed time.

Step 8: Stop early or run again (optional)

Press **Ctrl+C** to stop. The scanner prints a summary for the ports it has checked so far. To scan again, press the Up arrow to bring back `python port_scanner.py` and press Enter.

Part D: Ready-made demos

Demo 1: scan your own computer (safest, do this first)

Use two terminals, both inside the project folder.

Terminal 1: start a small web server on port 8000 and leave it running. The `--bind 127.0.0.1` part keeps it visible only to your own computer.

```
python -m http.server 8000 --bind 127.0.0.1
```

Terminal 2: run `python port_scanner.py` and enter `127.0.0.1`, then `7995`, then `8005`. The output should match the Step 7 example: port 8000 open with the banner `HTTP/1.0 200 OK`.

Optional: open `http://127.0.0.1:8000` in a browser to see the same server. **When finished:** click Terminal 1 and press `Ctrl+C`.

Demo 2: scan your home router

Find your router's address. On Windows run `ipconfig` and read **Default Gateway** (often `192.168.1.1` or `192.168.0.1`). On Mac or Linux, check your network settings. Run the scanner with that address, start port 20 and end port 80. Example output (yours will differ):

```
Scanning 192.168.1.1 (192.168.1.1), ports 20-80...
```

```
22/tcp  open  SSH-2.0-dropbear
23/tcp  open
53/tcp  open
80/tcp  open  HTTP/1.1 404 Not Found
```

```
Done: 4 open port(s) in 35.0s.
```

Port	Service	What the example shows
22	SSH	The banner shows Dropbear SSH.
23	Telnet	Unencrypted. Consider disabling it in the device's settings unless you turned it on on purpose.
53	DNS	A DNS-related service is listening on TCP port 53.
80	HTTP	An HTTP service is listening on port 80.

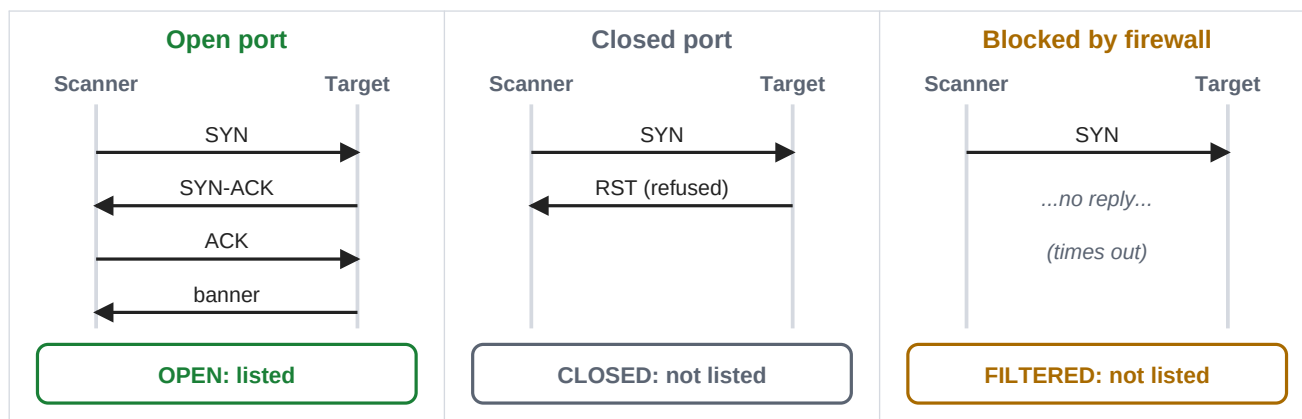
Demo 3 (optional): the public practice server

`scanme.nmap.org` is a server the Nmap project provides for practice scans. Enter it as the target with start port 80 and end port 443. It takes about 3 minutes and should show port 80 open. Scan it sparingly.

Part E: How the project works on a network

What happens when you press Enter

- 1. DNS lookup.** A name such as scanme.nmap.org is turned into an IP address once, at the start.
- 2. TCP connection to each port.** For every port the scanner asks the operating system to open a normal connection. The reply decides the result (diagram below).
- 3. Banner grab.** Services such as SSH, FTP and SMTP speak first, so the scanner just reads. Web servers wait for a request, so it sends a small HEAD / HTTP/1.0 request and reads the reply.
- 4. Summary.** Open ports and the elapsed time are printed.



SYN = "can we connect?", SYN-ACK = "yes", ACK = "connected", RST = "no, refused". The scanner cannot tell a closed port from a firewalled one, so neither is listed.

Which network layers it uses

Layer	What the scanner does there
Application (DNS, SSH, HTTP)	Resolves names, reads banners and sends the small HTTP request.
Transport (TCP ports)	Connects to each port number. The three-way handshake decides open or closed.
Network (IP)	Sends every packet to the target's IP address, on your local network or across the internet.

Proof that it works well

These are results from real test runs:

Target	Network	Result
A test web server on this computer (127.0.0.1)	Same computer	Found port 8000 with the banner HTTP/1.0 200 OK (11 ports in about 6 seconds).
A home router (192.168.1.1)	Home network	Found ports 22, 23, 53 and 80, with SSH and HTTP banners.
scanme.nmap.org	Internet	Found port 80 with the banner HTTP/1.1 200 OK (364 ports in about 3 minutes).
The project's 10 unit tests	Same computer	All pass: banner reading, the HTTP probe, silent services, closed ports and input checking.

Why it works well

- **Accurate.** A port is listed as open only after a full TCP connection succeeds, so it does not invent results.
- **Informative.** Banners show which software answers, not just a port number.
- **Gentle.** It makes one normal connection at a time, needs no administrator rights and changes nothing on the target.
- **Portable.** It uses only Python's standard library. It was tested on Windows; it should also run on Mac and Linux.
- **Tested and readable.** Ten automated tests pass, and the script is about 100 lines long.

What it is useful for

- Seeing what your own computers and router expose (for example, an open Telnet port).
- Checking that a firewall rule works or that a service is really running.
- Learning how ports, TCP connections, DNS and banners work.

Its limits

- It checks one port at a time, so wide ranges are slow.
- TCP and IPv4 only (no UDP, no IPv6).
- It cannot tell closed ports from firewalled ones, and it does not decode HTTPS banners.
- Its connection attempts appear in the target's logs.

Part F: Troubleshooting

Problem	Fix
python is not recognized	Try <code>py port_scanner.py</code> . On Mac and Linux use <code>python3</code> . Or reinstall Python with Add python.exe to PATH ticked, then open a new terminal.
Can't open file <code>port_scanner.py</code> (no such file)	The terminal is in the wrong folder. Run <code>dir</code> (or <code>ls</code>) and check that <code>port_scanner.py</code> is listed; use <code>cd</code> to go to the project folder.
Could not resolve 'name'.	The host name could not be turned into an IP address (a DNS lookup failure). Check the spelling and your network or internet connection, or enter the IP address instead.
The scan seems stuck	It is not. Each closed port can take up to 0.5 seconds. Use a smaller port range, or press <code>Ctrl+C</code> .
No open ports found	Nothing in that range is listening, or a firewall blocks it. Try Demo 1 to confirm the scanner works.
Known open ports are missing on a distant host	Increase <code>CONNECT_TIMEOUT</code> at the top of <code>port_scanner.py</code> (0.5 seconds by default).
Address already in use when starting the demo server	Something else uses port 8000. Start the server with 8001 instead (<code>python -m http.server 8001 --bind 127.0.0.1</code>). The same 7995 to 8005 scan still covers it.

Remember: use the scanner only on computers and devices you own or have explicit permission to scan.